

Advanced Computer Architecture Lab 2 : VLIW Processor Scheduler Simulation

LAP



Outline

- Processor & Instruction set overview
- Dependency table
- Static Scheduling without loop pipelining
 - Instruction Scheduling
 - Register renaming
- Initiation Interval
- Static Scheduling with loop pipelining
 - Instruction scheduling
 - Register renaming

Processor & Instruction set overview

High-level code snippet

```
for i in range(17):  
    listB[i] = listA[i] + listA[(i + 1) % 17]
```

Compiled RISC-V code



```
00  mov LC, 16  
01  mov x2, 0x1000  
02  mov x3, 0x2000  
03  ld x4, 0(x2)  
04  ld x5, 1(x2)  
05  add x6, x4, x5  
06  st x6, 0(x3)  
07  addi x2, x2, 1  
08  addi x3, x3, 1  
09  loop 03  
10  mov x7, 0x2000  
11  ld x2, 0(x3)  
12  ld x4, 0(x7)  
13  add x6, x2, x4  
14  st x6, 0(x3)
```

Instruction type	Latency	Execution Unit
mov	1	ALU
add	2	ALU
addi	2	ALU
ld	2	Memory
st	2	Memory
loop	1	Branch

Static scheduling → Scheduling at compile time

Dependencies

Three types of dependencies

- Read after Write (RaW) → True dependency
 - Write after Read (WaR) → False dependency
 - Write after Write (WaW) → False dependency
- These are not avoidable
- can be solved with more registers

WaR	addi x1, x2, 17	→	addi x1, x2, 17
	addi x2, x3, 51		addi x17, x3, 51
WaW	addi x4, x5, 17	→	addi x4, x5, 17
	addi x4, x6, 17		addi x18, x6, 17

True dependencies

```
00  mov LC, 16
01  mov x2, 0x1000
02  mov x3, 0x2000
03  ld x4, 0(x2)
04  ld x5, 1(x2)
05  add x6, x4, x5
06  st x6, 0(x3)
07  addi x2, x2, 1
08  addi x3, x3, 1
09  loop 03
10  mov x7, 0x2000
11  ld x2, 0(x3)
12  ld x4, 0(x7)
13  add x6, x2, x4
14  st x6, 0(x3)
```

Initialization

Loop

End

Local dependency :

03 : ld **x4**, 0(x2)

06 : add x6, **x4**, x5

Post loop dependency :

08 : addi **x3**, x3, 1

11 : ld x2, 0(**x3**)

Interloop dependency :

03 : ld x4, 0(**x2**)

07 : addi **x2**, x2, 1

Local : RaW in Initialization, in Loop or in End

Interloop : RaW in between 2 iterations of Loop

Loop invariant : RaW in between Loop and Initialization

Post loop : RaW in between End and Loop or Initialization

Dependency table

Instruction address	ID	Instruction	Destination register	True dependencies			
				Local	Interloop	Loop Invariant	Post Loop
0	A	mov LC, 16	LC	-	-	-	-
1	B	mov x2, 0x1000	x2	-	-	-	-
2	C	mov x3, 0x2000	x3	-	-	-	-
3	D	ld x4, 0(x2)	x4	-	x2 : (B, H')	-	-
4	E	ld x5, 1(x2)	x5	-	x2 : (B, H')	-	-
5	F	add x6, x4, x5	x6	x4 : D / x5 : E	-	-	-
6	G	st x6, 0(x3)	-	x6 : F	x3 : (C, I')	-	-
7	H	addi x2, x2, 1	x2	-	x2 : (B, H')	-	-
8	I	addi x3, x3, 1	x3	-	x3 : (C, I')	-	-
9	J	loop 3	LC	-	LC : (A, J')	-	-
10	K	mov x7, 0x2000	x7	-	-	-	-
11	L	ld x2, 0(x3)	x2	-	-	-	x3 : I
12	M	ld x4, 0(x7)	x4	x7 : K	-	-	-
13	N	add x6, x2, x4	x6	x2 : L / x4 : M	-	-	-
14	O	st x6, 0(x3)	-	x6 : N	-	-	x3 : I

Instruction D - x2 : (B, H') → H writes x2 (iteration i-1), D reads x2 (iteration i)

Instruction scheduling

1st step : Handle false dependencies → Allocate new destination registers

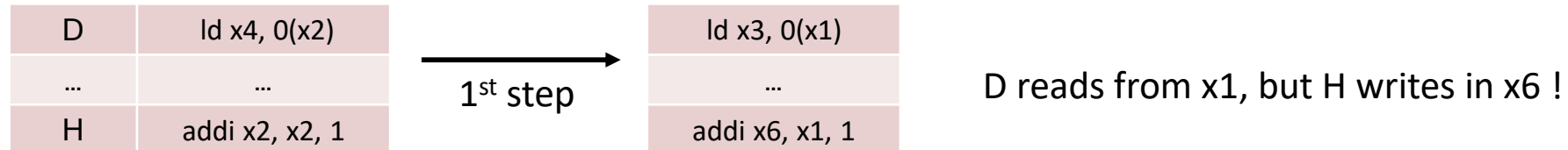
Instruction	Destination register
mov LC, 16	LC
mov x2, 0x1000	x2
mov x3, 0x2000	x3
ld x4, 0(x2)	x4
ld x5, 1(x2)	x5
add x6, x4, x5	x6
st x6, 0(x3)	-
addi x2, x2, 1	x2
addi x3, x3, 1	x3
loop 3	LC
mov x7, 0x2000	x7
ld x2, 0(x3)	x2
ld x4, 0(x7)	x4
add x6, x2, x4	x6
st x6, 0(x3)	-



New destination register	New Instruction
LC	mov LC, 16
x1	mov x1, 0x1000
x2	mov x2, 0x2000
x3	ld x3, 0(x1)
x4	ld x4, 1(x1)
x5	add x5, x3, x4
-	st x5, 0(x2)
x6	addi x6, x1, 1
x7	addi x7, x2, 1
LC	loop 3
x8	mov x8, 0x2000
x9	ld x9, 0(x7)
x10	ld x10, 0(x8)
x11	add x11, x9, x10
-	st x11, 0(x7)

Instruction scheduling

2nd step : Fix interloop dependencies → broken during the first step



ID	Interloop
D	x2 : (B, H')
E	x2 : (B, H')
F	-
G	x3 : (C, I')
H	x2 : (B, H')
I	x3 : (C, I')
J	LC : (A, J')

2 registers : x2 and x3

LC is ignored (only changed by loop instruction)

After new register allocation :

x2 → x1

x3 → x2

x2 last written by H → x2 is now x1, H writes in x6, thus : **mov x1, x6**

x3 last written by I → x3 now x2, I writes in x7, thus : **mov x2, x7**

we do not ignore the interloop dependency semantically,

we just do not need to schedule it explicitly because the non-pipelined loop boundary already enforces it

: In a non-pipelined loop, iteration i+1 cannot start until iteration i has completely finished.

So the rule is:

old destination register → new unique destination register



Example:

Instruction	Old destination	New destination
B: mov x2, 0x1000	x2	x1
C: mov x3, 0x2000	x3	x2
D: ld x4, 0(x2)	x4	x3
E: ld x5, 1(x2)	x5	x4
F: add x6, x4, x5	x6	x5
H: addi x2, x2, 1	x2	x6
I: addi x3, x3, 1	x3	x7



Instruction scheduling

3rd step : Schedule the instructions --> Worry only about local, loop invariant and post loop dependencies

Schedule D (ld x3, 0(x1)) : Start of Loop → Earliest possible (after Initialization)

B scheduled at 0 → x1 ready at 1 (mov latency : 1)

Initialization

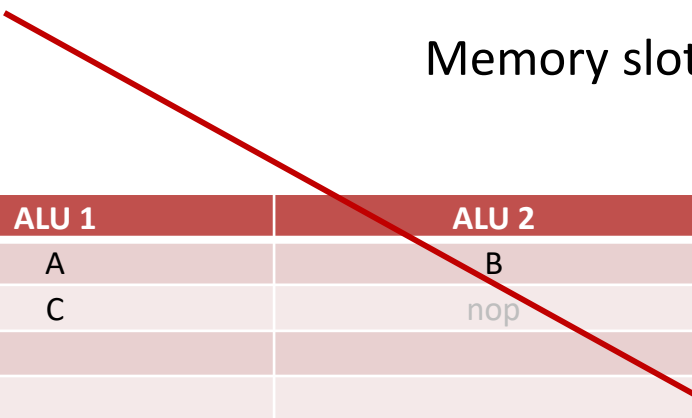
Loop

	ALU 1	ALU 2	Memory Unit	Branch Unit
0	A	B	nop	nop
1	C	nop	nop	nop
2			D	
3				
4				
5				
6				
7				
8				
9				
10				
11				
12				
13				

Instruction scheduling

Schedule E (ld x4, 1(x1)) : Earliest possible (same, x1 ready since 1)

Memory slot busy at 2 → E scheduled in slot 3



	ALU 1	ALU 2	Memory Unit	Branch Unit
0	A	B	nop	nop
1	C	nop	nop	nop
2			D	
3			E	
4				
5				
6				
7				
8				
9				
10				
11				
12				
13				

Instruction scheduling

Schedule F (add x5, x3, x4) : x3 is written by D, x4 by E

Latency of ld : 2 $\rightarrow$ x3 ready at 4, x4 at 5



	ALU 1	ALU 2	Memory Unit	Branch Unit
0	A	B	nop	nop
1	C	nop	nop	nop
2			D	
3			E	
4				
5	F			
6				
7				
8				
9				
10				
11				
12				
13				

Static schedule without loop pipelining

	ALU 1	ALU 2	Memory Unit	Branch Unit
0	A	B	nop	nop
1	C	nop	nop	nop
2	H	I	D	nop
3	nop	nop	E	nop
4	nop	nop	nop	nop
5	F	nop	nop	nop
6	nop	nop	nop	nop
7	Y	Z	G	J : loop 2
8	K	nop	L	nop
9	nop	nop	M	nop
10	nop	nop	nop	nop
11	N	nop	nop	nop
12	nop	nop	nop	nop
13	nop	nop	O	nop

Loop pipelining

Example code :

0 : A

1 : B

2 : C

3 : D

A = (add, add, nop, nop)
B = (add, nop, st, nop)
C = (nop, nop, ld, nop)
D = (nop, add, nop, loop)

Without loop pipelining

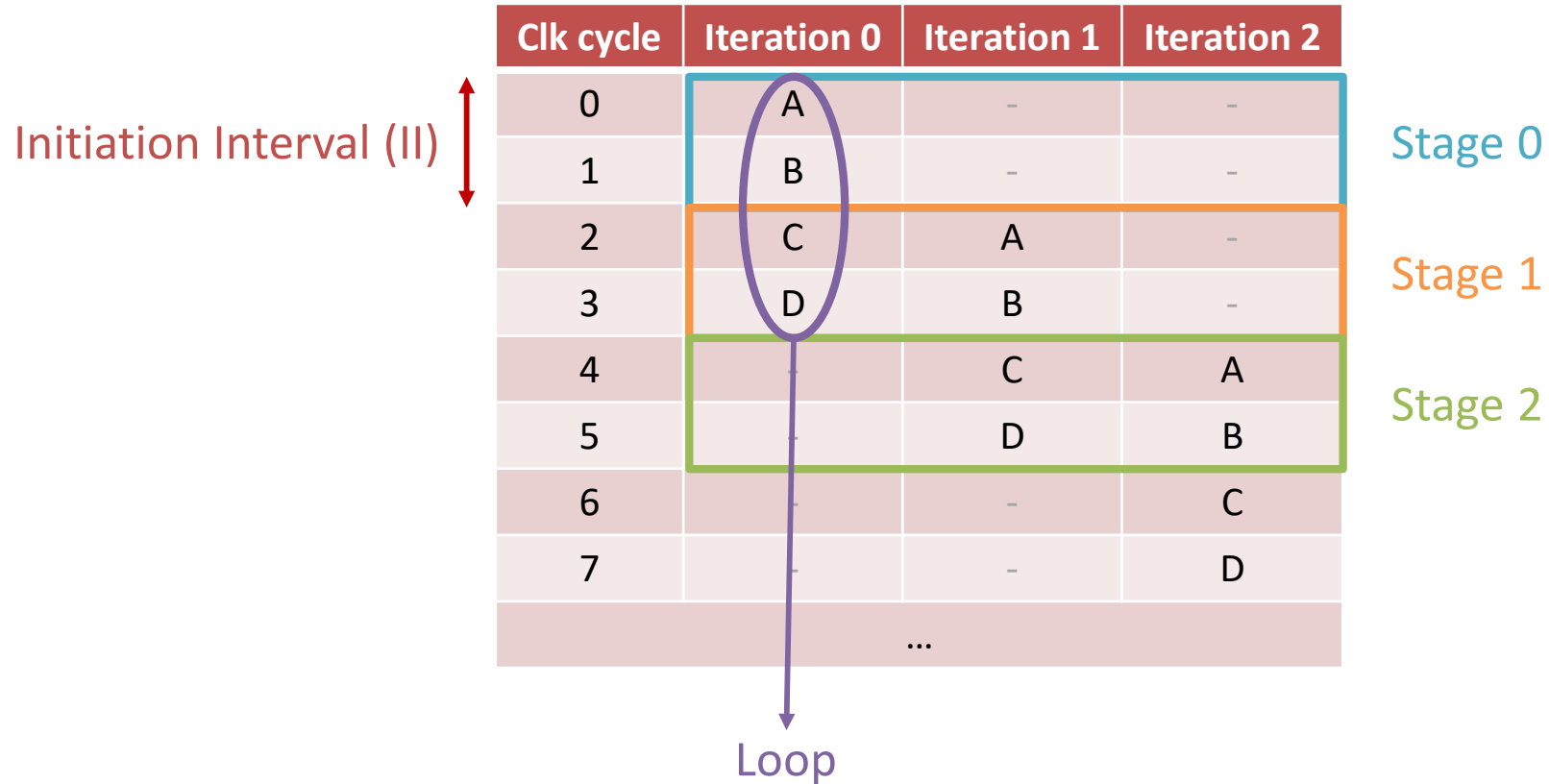
Clk cycle	Iteration 0	Iteration 1	Iteration 2
0	A	-	-
1	B	-	-
2	C	-	-
3	D	-	-
4	-	A	-
5	-	B	-
6	-	C	-
7	-	D	-
8	-	-	A
9	-	-	B
...			

With loop pipelining

Clk cycle	Iteration 0	Iteration 1	Iteration 2
0	A	-	-
1	B	-	-
2	C	A	-
3	D	B	-
4	-	C	A
5	-	D	B
6	-	-	C
7	-	-	D
...			

0 : (add, add, nop, nop)
1 : (add, nop, st, nop)
2 : (add, add, ld, nop)
3 : (add, add, st, loop)
4 : (add, add, ld, nop)
5 : (add, add, st, loop)
...

Loop pipelining



/!\ In this lab, we ignore the last step (how to handle the last iteration)

Initiation Interval

```
00  mov LC, 16
01  mov x2, 0x1000
02  mov x3, 0x2000
03  ld x4, 0(x2)
04  ld x5, 1(x2)
05  add x6, x4, x5
06  st x6, 0(x3)
07  addi x2, x2, 1
08  addi x3, x3, 1
09  loop 03
10  mov x7, 0x2000
11  ld x2, 0(x3)
12  ld x4, 0(x7)
13  add x6, x2, x4
14  st x6, 0(x3)
```

$$II \geq \left\lceil \frac{N}{U} \right\rceil$$

Units	ALU	Memory	Branch
N	3	3	1
U	2	1	1
N/U	2	3	1

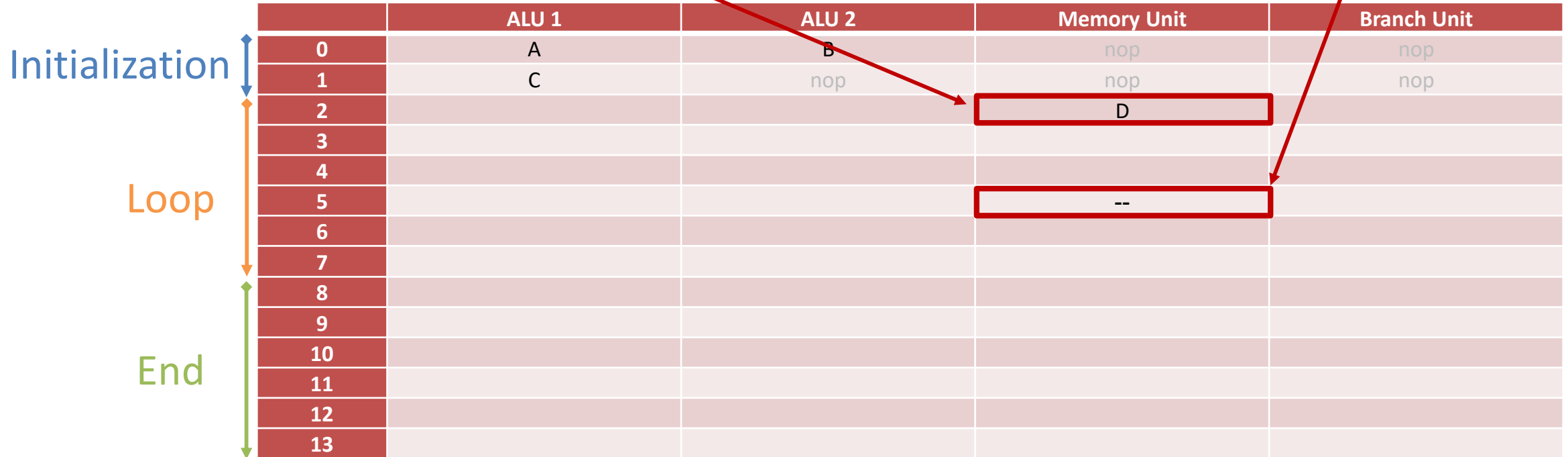
Bottleneck

First try : $II = 3$

Instruction scheduling

Schedule D (ld x4, 0(x2)) : Start of Loop → Earliest possible (after Initialization)

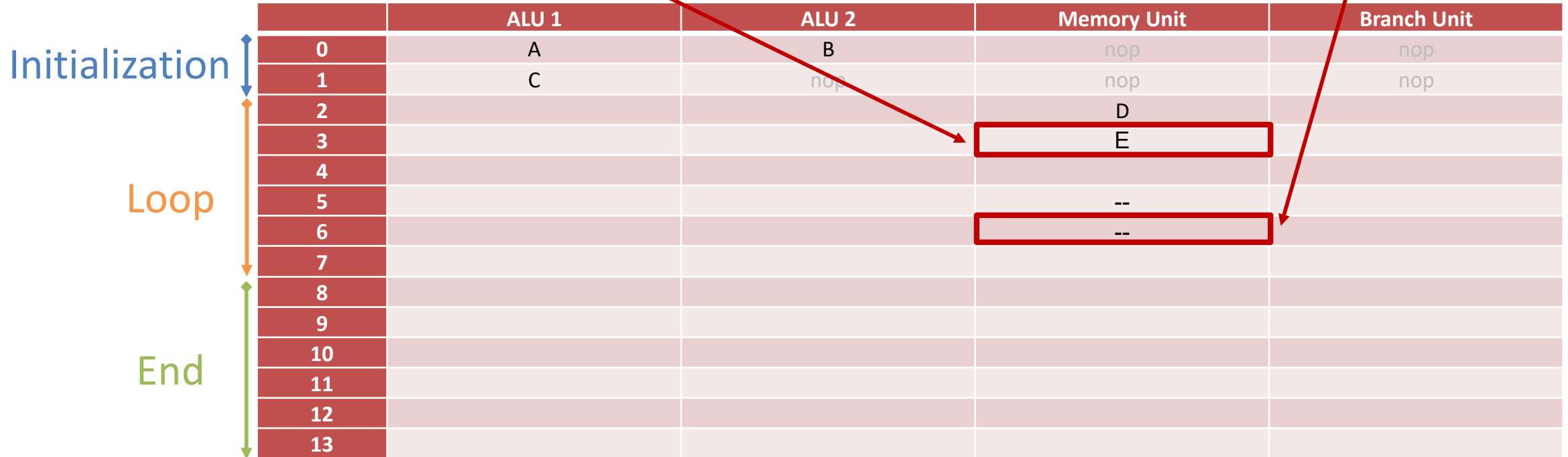
II = 3 → Reserve the unit at every third clock cycles



Instruction scheduling

Schedule E (ld x5, 0(x2))

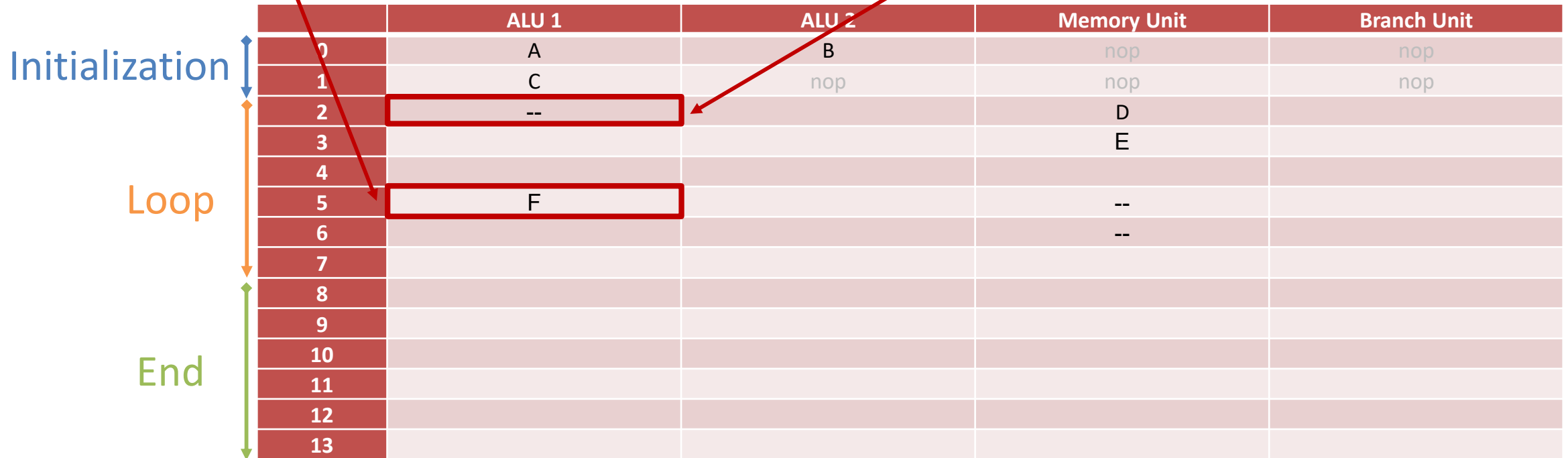
II = 3 $\rightarrow$ Reserve the unit at every third clock cycles



Instruction scheduling

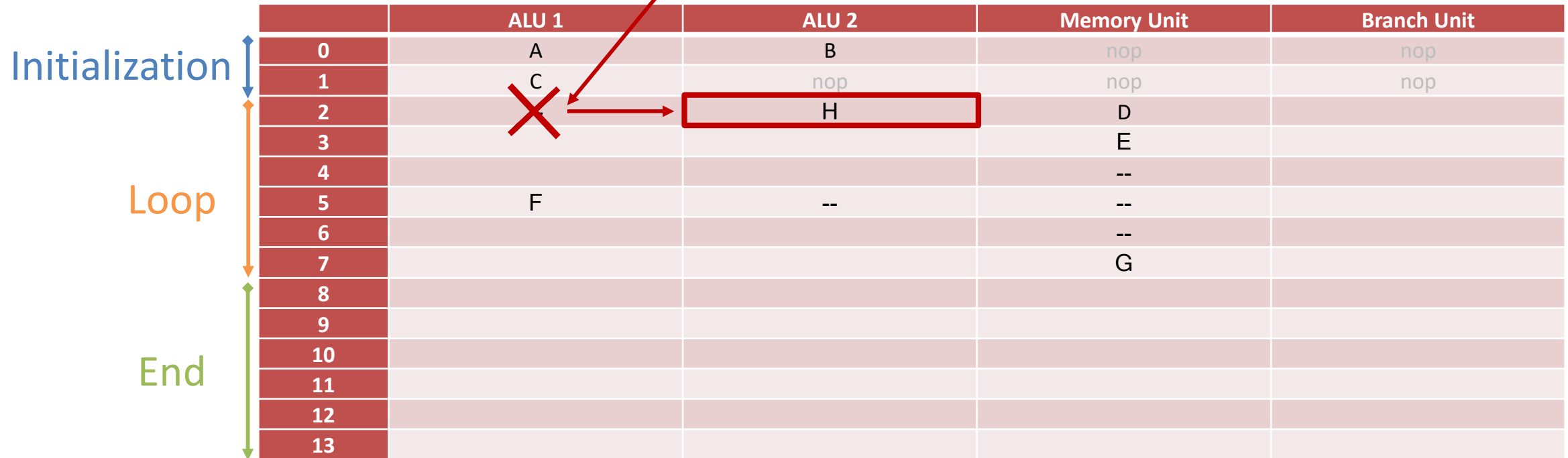
Schedule F (add x6, x4, x5) : x4 is written by D (ready at 4), x5 by E (ready at 5)

II = 3 $\rightarrow$ Reserve the unit at every third clock cycles



Instruction scheduling

Schedule H → addi x2, x2, 1



Static schedule with loop pipelining

	ALU 1	ALU 2	Memory Unit	Branch Unit
0	A	B	nop	nop
1	C	nop	nop	nop
2	--	H	D	nop
3	I	nop	E	nop
4	nop	nop	--	J : loop 2
5	F	--	--	nop
6	--	nop	--	nop
7	nop	nop	G	--
8	K	nop	L	nop
9	nop	nop	M	nop
10	nop	nop	nop	nop
11	N	nop	nop	nop
12	nop	nop	nop	nop
13	nop	nop	O	nop

Issue : Loop dependencies are broken !

Solution : Allocate different registers to different loop iterations
→ Rotating registers

At each new stage, shift every register by 1

More precisely : $x_{\text{current}} = x_{\text{base}} + (\text{current_iteration} \% (\text{loop_length} / II))$

x_{base} → register written in the static schedule (before any shift happens)

$$x_{\text{base}} = x(\textcolor{green}{Z} + \textcolor{blue}{a} + \textcolor{brown}{b})$$

$\textcolor{green}{Z}$: Start register ($\approx$ the one if no shifts where operated)

$\textcolor{blue}{a}$: Iteration offset → Handle interloop dependencies

$\textcolor{brown}{b}$: Stage Offset → Handle inner loop dependencies

Register allocation (Loop)

Allocate a new destination register xZ to every loop instruction

Leave enough space between each xZ to perform the shifts

→ # stages per iteration + 1

Rotating registers start at x32

stages per iteration = 2 → x32, x35, x38, ...

ID	Old Instruction	Old Destination Register	Base Rotating Register
D	ld x4, 0(x2)	x4	x32
E	ld x5, 1(x2)	x5	x35
F	add x6, x4, x5	x6	x38
H	addi x2, x2, 1	x2	x41
I	addi x3, x3, 1	x3	x44

Note that x2 and x3 (destination registers of B and C) are allocated !

Register allocation (outside the loop)

Allocate a new destination register to other instructions

Others -> Local non-loop dependencies + Loop invariant

The regular x1, x2, x3, ... are used

ID	Old Instruction	Old Destination Register	New Destination Register
K	mov x7, 0x2000	x7	x1
L	ld x2, 0(x3)	x2	x2
M	ld x4, 0(x7)	x4	x3
N	add x6, x2, x4	x6	x4

Offset assignment

B : mov x41(+a, +b), 0x1000

		ALU 1	ALU 2	Memory Unit	Branch Unit
Initialization	0	A	B	nop	nop
	1	C	nop	nop	nop
Stage 0	2	--	H	D	nop
	3	I	nop	E	nop
Stage 1	4	nop	nop	--	J : loop 2
	5	F	--	--	nop
	6	--	nop	--	nop
	7	nop	nop	G	--
	8	K	nop	L	nop
	9	nop	nop	M	nop
End	10	nop	nop	nop	nop
	11	N	nop	nop	nop
	12	nop	nop	nop	nop
	13	nop	nop	O	nop

B in Initialization (special case) $\rightarrow a = 1, b = 0$

Offset assignment

D : ld x32, 0(x41(+1, +0))

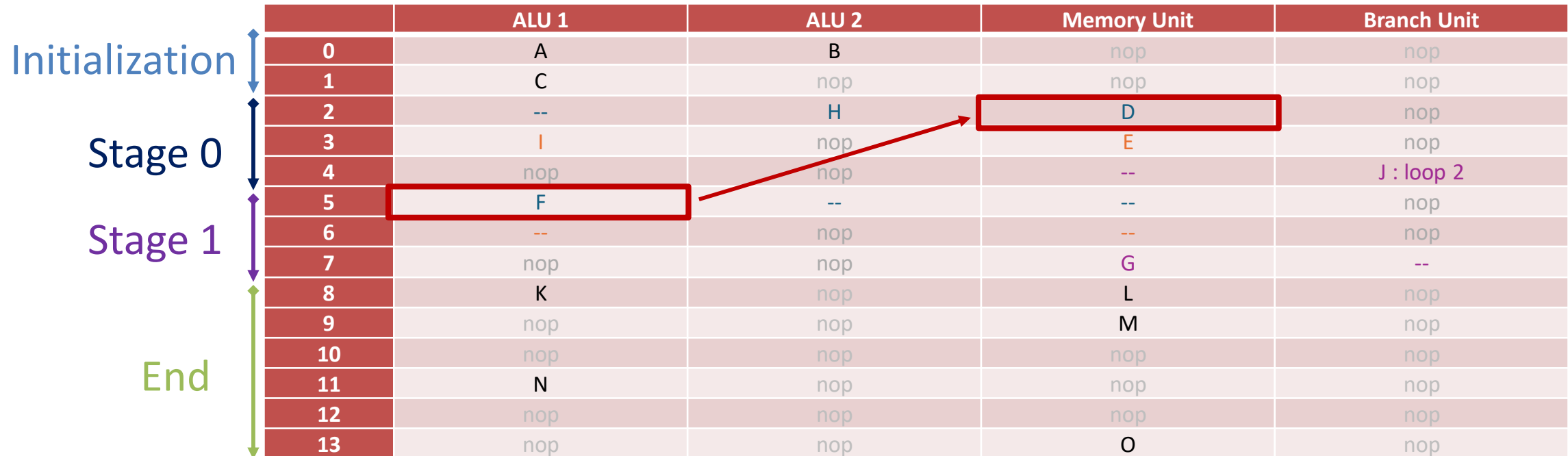
		ALU 1	ALU 2	Memory Unit	Branch Unit
Initialization	0	A	B	nop	nop
	1	C	nop	nop	nop
Stage 0	2	--	H	D	nop
	3	I	nop	E	nop
Stage 1	4	nop	nop	--	J : loop 2
	5	F	--	--	nop
End	6	--	nop	--	nop
	7	nop	nop	G	--
	8	K	nop	L	nop
	9	nop	nop	M	nop
	10	nop	nop	nop	nop
	11	N	nop	nop	nop
	12	nop	nop	nop	nop
	13	nop	nop	O	nop

D reads x41, x41 is written by H (and B) :

- comes from previous iteration (interloop dependency) $\rightarrow a = 1$
- comes from the same stage (H in stage 0) $\rightarrow b = 0$

Offset assignment

F : add x38, x32(+a, +b), x35(+?, +?)



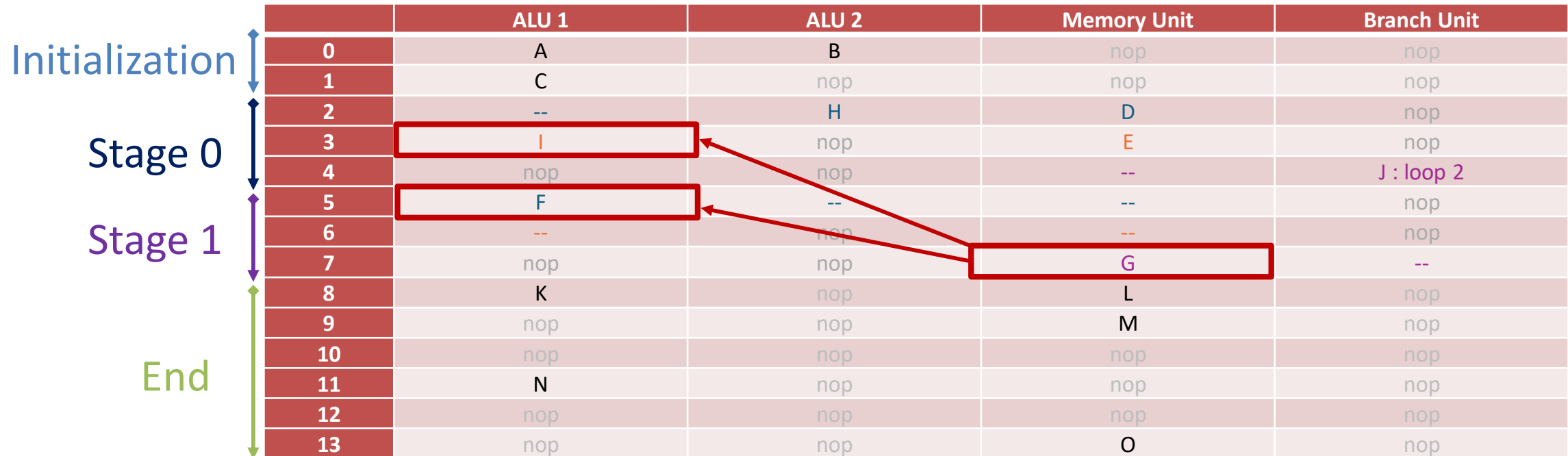
F reads x32, x32 is written by D :

- comes from the same iteration $\rightarrow a = 0$
- comes from the previous stage (D is in stage 0, F in stage 1) $\rightarrow b = 1 - 0 = 1$

Same reasoning for x35 !

Offset assignment

G : st x38(+a, +b), 0(x44(+a', +b'))



G reads x38, x38 is written by F :

- comes from the same iteration $\rightarrow a = 0$
- comes from the same stage $\rightarrow b = 0$

G reads x44, x44 is written by I :

- comes from the previous iteration $\rightarrow a' = 1$
- comes from the previous stage $\rightarrow b' = 1$

Offset assignment

L : ld x2, 0(x44(+a, +b))

		ALU 1	ALU 2	Memory Unit	Branch Unit
Initialization	0	A	B	nop	nop
	1	C	nop	nop	nop
Stage 0	2	--	H	D	nop
	3	I	nop	E	nop
Stage 1	4	nop	nop	--	J : loop 2
	5	F	--	--	nop
End	6	--	nop	--	nop
	7	nop	nop	G	--
	8	K	nop	L	nop
	9	nop	nop	M	nop
	10	nop	nop	nop	nop
	11	N	nop	nop	nop
	12	nop	nop	nop	nop
	13	nop	nop	O	nop

L in End (special case) $\rightarrow a = 0, b = 1$

Offset assignment

ID	Old Instruction	New Instruction
A	mov LC, 16	mov LC, 16
B	mov x2, 0x1000	mov x41(+1, +0), 0x1000
C	mov x3, 0x2000	mov x44(+1, +0), 0x2000
D	ld x4, 0(x2)	ld x32, 0(x41(+1, +0))
E	ld x5, 1(x2)	ld x35, 1(x41(+1, +0))
F	add x6, x4, x5	add x38, x32(+0, +1), x35(+0, +1)
G	st x6, 0(x3)	st x38(+0, +0), 0(x44(+1, +1))
H	addi x2, x2, 1	addi x41, x41(+1, +0), 1
I	addi x3, x3, 1	addi x44, x44(+1, +0), 1
J	loop 3	loop 2
K	mov x7, 0x2000	mov x1, 0x2000
L	ld x2, 0(x3)	ld x2, 0(x44(+0, +1))
M	ld x4, 0(x7)	ld x3, 0(x1)
N	add x6, x2, x4	add x4, x2, x3
O	st x6, 0(x3)	st x4, 0(x44(+0, +1))

Static schedule with loop pipelining

	ALU 1	ALU 2	Memory Unit	Branch Unit
0	mov LC, 16	mov x42, 0x1000	nop	nop
1	mov x45, 0x2000	nop	nop	nop
2	--	addi x41, x42, 1	ld x32, 0(x42)	nop
3	addi x44, x45, 1	nop	ld x35, 1(x42)	nop
4	nop	nop	--	J : loop 2
5	add x38, x33, x36	--	--	nop
6	--	nop	--	nop
7	nop	nop	st x38, 0(x46)	--
8	mov x1, 0x2000	nop	ld x2, 0(x45)	nop
9	nop	nop	ld x3, 0(x1)	nop
10	nop	nop	nop	nop
11	add x4, x2, x3	nop	nop	nop
12	nop	nop	nop	nop
13	nop	nop	st x4, 0(x45)	nop